

Image Classification by Transfer Learning using Pre-Trained CNN Models

Jaya H. Dewan

Department of Information Technology
Pimpri Chinchwad College of
Engineering
Pune, India
jaya.h.dewan@gmail.com

Rik Das

ACM Professional Member
Association for Computing Machinery
Ranchi, India
rikdas78@gmail.com

Sudeep D. Thepade

Computer Engineering Department
Pimpri Chinchwad College of
Engineering
Pune, India
sudeepthepade@gmail.com

Harsh Jadhav

Department of Information Technology
Pimpri Chinchwad College of
Engineering
Pune, India
harsh.ajay.jadhav@gmail.com

Nishant Narsale

Department of Information Technology
Pimpri Chinchwad College of
Engineering
Pune, India
nishantnarsale510@gmail.com

Ajinkya Mhasawade

Department of Information Technology
Pimpri Chinchwad College of
Engineering
Pune, India
deutscherajinkya@gmail.com

Sinu Nambiar

Department of Computer Engineering
D.Y.Patil Institute of Technology
Pune, India
sinunambiar@gmail.com

Abstract—In the realm of computer vision, image classification is a critical issue with many applications, including multimedia content analysis, security and surveillance, and medical imaging. The accuracy of image classification algorithms has considerably increased with the development of deep learning. The discipline of image classification has dramatically benefited from the usage of pre-trained Convolutional Neural Network (CNN) models. In this study, we perform image classification on the Wang dataset composed of 1000 images separated into ten categories, using six different pre-trained CNN models. The research made use of pre-trained versions of VGG16, Densenet, Mobilenet, Inception V3, Resnet50, and Xception models. We assessed the model performances in terms of training and testing accuracy. Testing accuracies for ten unique batches of data were calculated and averaged out. Densenet outperformed state-of-the-art models like Xception by a small margin, mainly due to low data availability. The findings of this study show how pre-trained models have advanced the field of image classification and shed light on how well these models perform in diverse image classification tasks. This study can serve as a valuable reference for researchers and practitioners in the field of computer vision and deep learning and help inform their choices when selecting pre-trained models for image classification, depending on their needs, while also considering data availability and computational constraints.

Keywords—Transfer Learning, Image Classification, Deep Convolutional Neural Networks

I. INTRODUCTION

Images are one of the most fundamental ways of capturing and storing information. They are able to store visual information such as color, depth, distance, and brightness more efficiently than the standard tabular way of storing data. As such, the creation of tools that can effectively extract that information and make concrete predictions based on it is vital. Image classification is the action of extracting data from an image in order to assign it a specific label or class. It has been a widely studied domain of interest in recent years. The importance of image classification in computer vision has been growing at an exponential rate, mainly

because it finds its use in a wide array of applications such as object recognition, quality control, medical diagnosis, content-based image retrieval, and video analysis [1][2][3][4]. Deep Convolutional Neural Networks (CNNs) have become a popular solution for this problem due to their ability to learn high-level representations of images. In principle, Artificial Neural Networks (ANNs) can be used for image classification tasks, but CNNs are able to achieve the same goal in a less computationally intensive way while also giving better results in general, hence they are popular. In many cases, training a deep CNN from scratch is a time-consuming and computationally expensive task, especially when dealing with large datasets. To mitigate this issue, researchers have proposed using pre-trained deep CNN models, which have already been trained on large datasets and can be fine-tuned on smaller datasets for specific image classification tasks.

In this research, the use of pre-trained deep CNN models for image classification are explored. The main objective is to investigate the impact of pre-trained deep CNN models on the performance and computational efficiency of image classification tasks. The performance comparison of different pre-trained deep CNN models and fine-tuning strategies on benchmark datasets are presented.

II. RELATED WORK

CNNs show great promise for feature extraction from images. Furthermore, the use of pre-trained models and transfer learning of these models has also made it possible to increase performance. These methods find use in image classification tasks in many fields like medicine, facial recognition, object recognition, and so on.

Transfer learning with CNNs has been researched widely. The CNN model with VGG, ResNet, and DenseNet is used to detect plant diseases in [5]. In [6], transfer learning of VGG, ResNet, and Inception are used to classify brain tumors, and 100% testing data accuracy was seen. In a study to predict cell responses to chemical perturbation, ResNet, InceptionV3, and Inception ResnetV2 with transfer learning were utilized. They provided 97%, 97%, and 95% mean

accuracy, respectively [7]. [8] uses transfer learning with 11 pre-trained models for the purpose of detecting malignancies, with AlexNet achieving 100% accuracy. In research on facial emotion recognition, VGG16 achieved 98.33% accuracy compared to 95% of Alexnet [9]. [10] contrasted CNN and Resnet models in order to categorize ages using facial features.

Much study has been done for pre-trained and non-pre-trained models in image feature extraction and object recognition. With the CIFAR dataset, one study examined CNN, KNN, SVM, Softmax, and Fully Connected networks, with CNN performing better [11]. In research comparing a model built on the ImageNet dataset, the simple CNN scored 46% accurately, whereas the pre-trained model scored 87% [12]. A CNN-SVM hybrid model and transfer learning on the VGG16, VGG19, and AlexNet were compared in a study. VGG19 outperformed the competition [13].

Here, a comparative analysis of six pre-trained models for generic image classification has been performed.

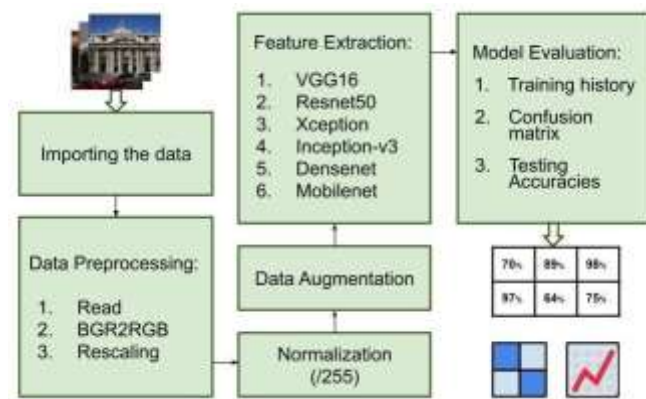


Fig. 1. Block Diagram of the Proposed System

III. PROPOSED SYSTEM

Fig. 1 demonstrates the steps involved in the proposed system. The following steps were followed:

1. Importing the data: Wang dataset was imported from Kaggle and processed in a jupyter notebook environment.
2. Data preprocessing: The files were iterated to perform some necessary functions. The labels are extracted from the file names, and the files are read using the OpenCV library. OpenCV reads the images in BGR format. They were converted to RGB format and resized to 256x256 pixels to maintain uniformity and reduce the amount of data fed into the models. The images and the labels were stored in separate empty lists and sent for further processing.
3. Normalization: The image list was converted to a numpy array, and the pixel values were divided by 255 to fit them between a scale of 0 to 1.
4. Data augmentation: The ImageDataGenerator class from the Keras library was used to augment the data. This was done to diversify the features encountered in the images as well as increase the size of the dataset, which in turn helped the models to show better performance. Operations like width shift, height shift, horizontal flip, and vertical flip were performed.

5. Feature extraction using transfer learning: models pre-trained on the imagenet dataset were downloaded from the Keras library. The parameter `include_top=False` was passed to download the whole model except the fully connected layer at the end, which were changed according to the number of categories in the data.

The following models were analyzed in this study.

A. VGG16

VGG16[14], as shown in Fig. 2, has a simple, straightforward, and uniform architecture consisting of 16 layers. 13 convolutional layers divided into five blocks (the number of filters varies from block to block), and three fully connected layers at the end. Takes in a 224x224x3 image as the input. 3x3 convolutional filters are used throughout the architecture, followed by a 2x2 max-pool at the end of each block. The fully connected layers have a total of 4096 channels, and the final layer (soft-max) classifies the image into ten classes according to the particular dataset.

B. Resnet50

Resnet50 is a 50-layer deep network architecture that uses skip connections allowing deeper networks to be trained without having to worry about increasing training loss[15]. It aims to solve the vanishing gradient problem encountered in deep learning. High-resolution outputs from the previous layers are directly passed to the deeper layers, which initially were only dealing with low-resolution spatial or contextual information. The skip connections, as shown in the architecture in Fig. 3, help in making the output at least equal to the input from the previous layers and reducing the effect of vanishing gradients during back-propagation. This allows us to train deeper models while maintaining lower loss levels.

C. Inception v3

The architecture of Inception v1, shown in Fig. 4, makes use of different filter sizes in the same convolutional block [16]. This ensures that features of multiple sizes can be extracted from the input volume. For example, it takes an input and applies 1x1, 2x2, 3x3, 5x5, and max-pool (same-padded) in a single block to give multiple features volumes having different depths. These volumes are then concatenated to give a final output that can be fed to the next layer. Computational complexity is reduced by performing 1x1 convolutions. They reduce the channel depth of the volumes without affecting the height and width. The simultaneous convolutions allow the network to go wider. Inception v3 reduces computational complexity even further by factorization into smaller and asymmetric convolutions.

D. Xception

Xception [17] has the same amount of trainable parameters as inception v3. It outperforms inception v3 on the imagenet dataset, suggesting great computational efficiency. It simplifies the inception architecture and uses skip connections and depth wise separable convolutions (pointwise convolutions followed by depthwise convolutions in the case of Xception). This helps in reducing the number of computations by a great number. Xception is divided into three modules, the entry flow, the middle flow, which is repeated eight times, and the exit flow. The architecture is shown in Fig. 5.

E. Densenet

Densenet, shown in Fig. 6 builds on the concepts of skip connections by sending the output to each succeeding layer in a block [18]. The passed feature maps are concatenated at the receiving layer. So, for a block having N layers, the total number of connections would be $N(N+1)/2$. Densenet has a number of dense blocks in which the connections are made compared to connecting all the layers of the model to one another, which would make it very complex. The Densenet-121, consisting of 121 layers and four dense blocks is used here. The first three dense blocks are followed by a transition layer, while the last dense block is followed by the classification layer. The transition layer applies a 1×1 convolution to reduce the channel depth, followed by an average pool.

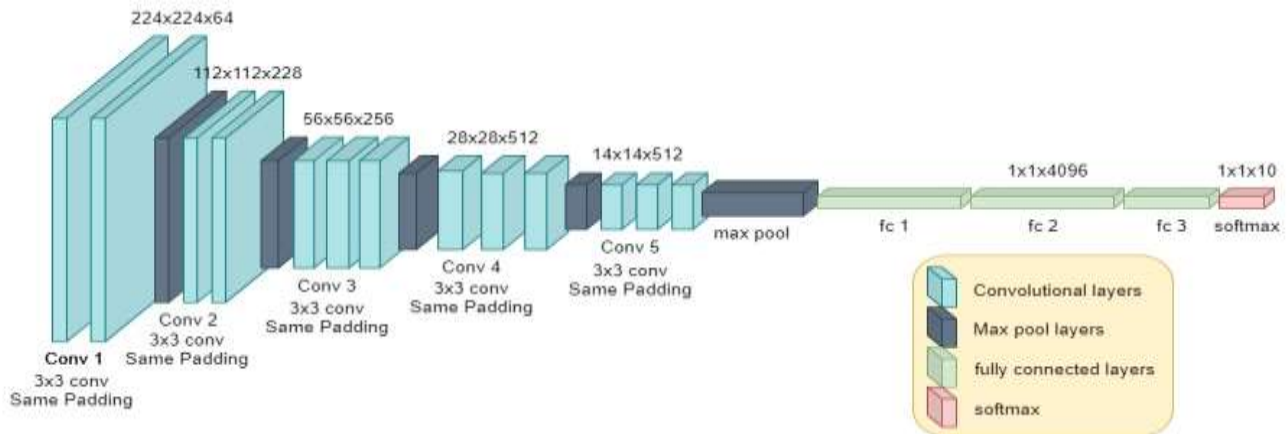


Fig. 2. VGG16 architecture

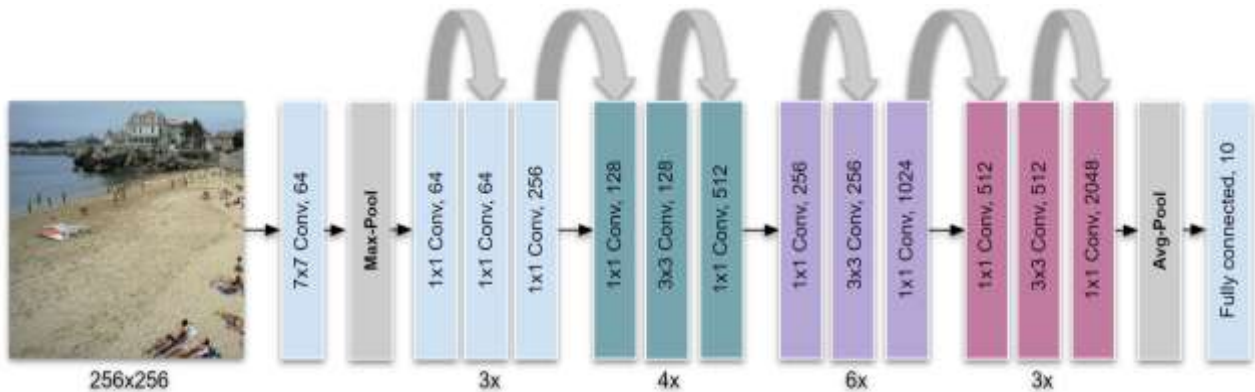


Fig. 3. Resnet50 architecture

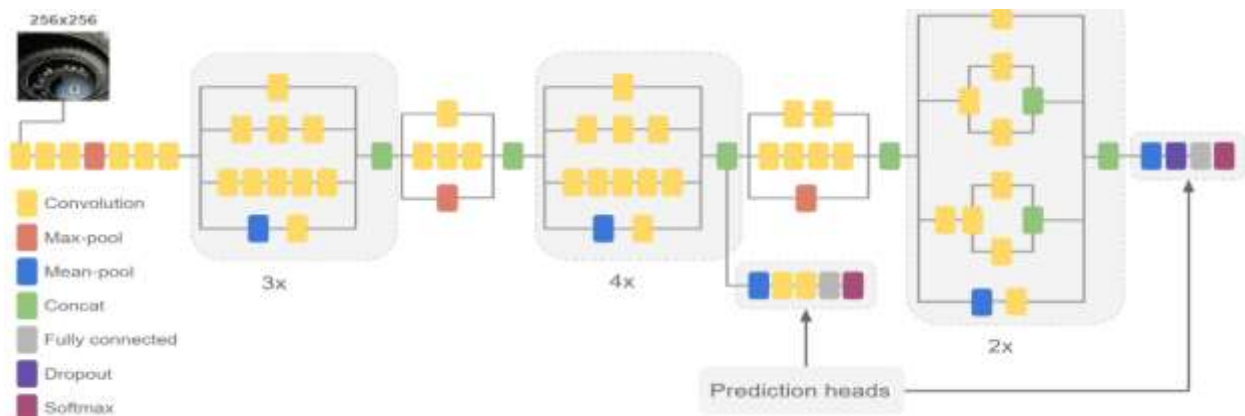


Fig. 4. Inception v3 architecture

F. Mobilenet

The Mobilenet [19] architecture shown in Fig. 7 has a very low number of parameters compared to other deep CNNs. Mobilenet, as the name suggests, is a light model developed for deployment into mobile applications where, storing and training a huge number of parameters is not possible. Mobilenet uses depthwise separable convolutions to reduce computational complexity. 2D convolutions are performed over the whole depth of the input volume, whereas depthwise separable convolutions are performed separately at each channel depth, followed by 1×1 (pointwise) convolution to save on cost. Mobilenet has to trade off on accuracy because of the small size, but the tradeoff is very small.

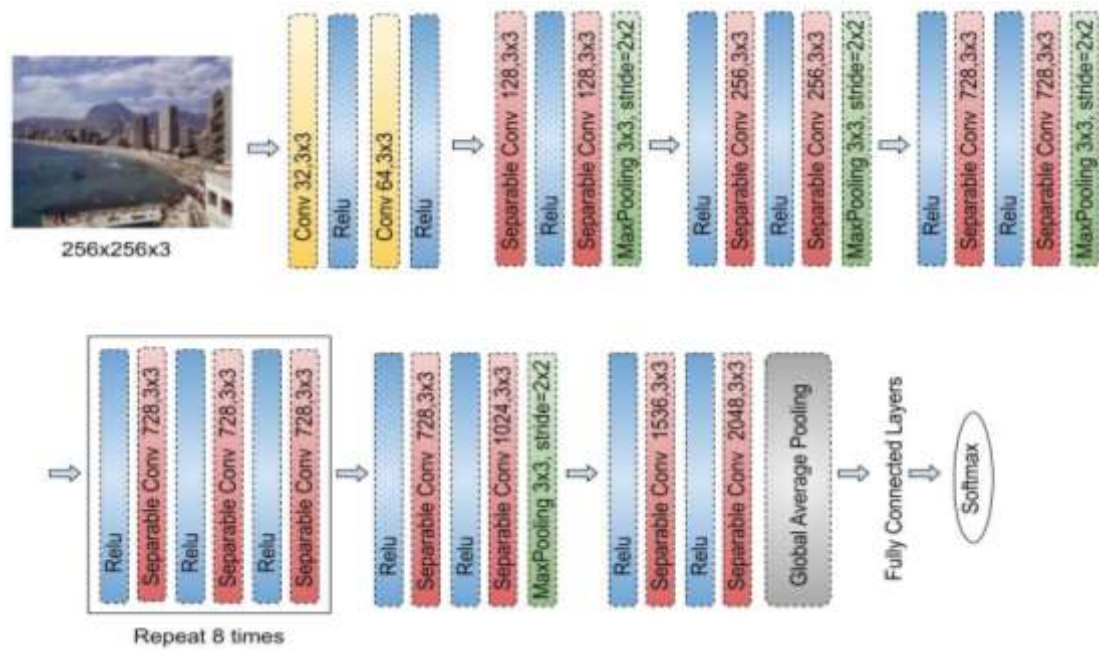


Fig. 5. Xception architecture

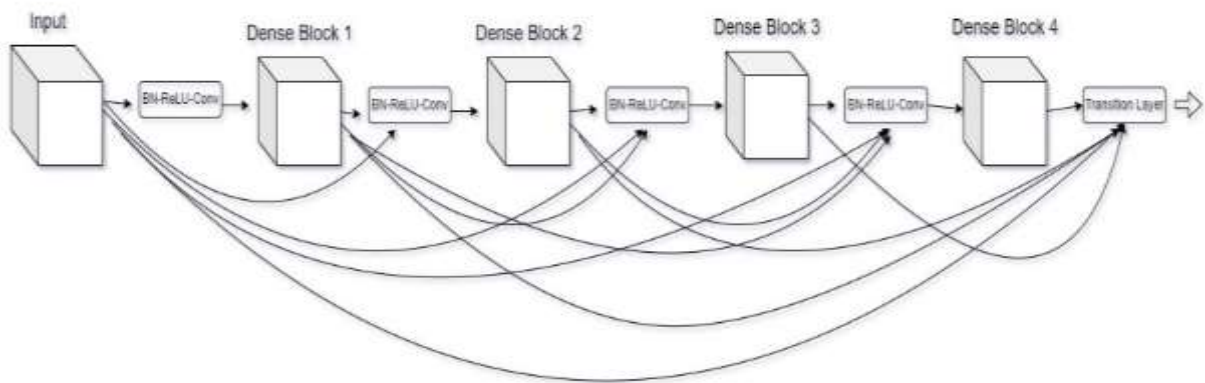


Fig. 6. Densenet architecture

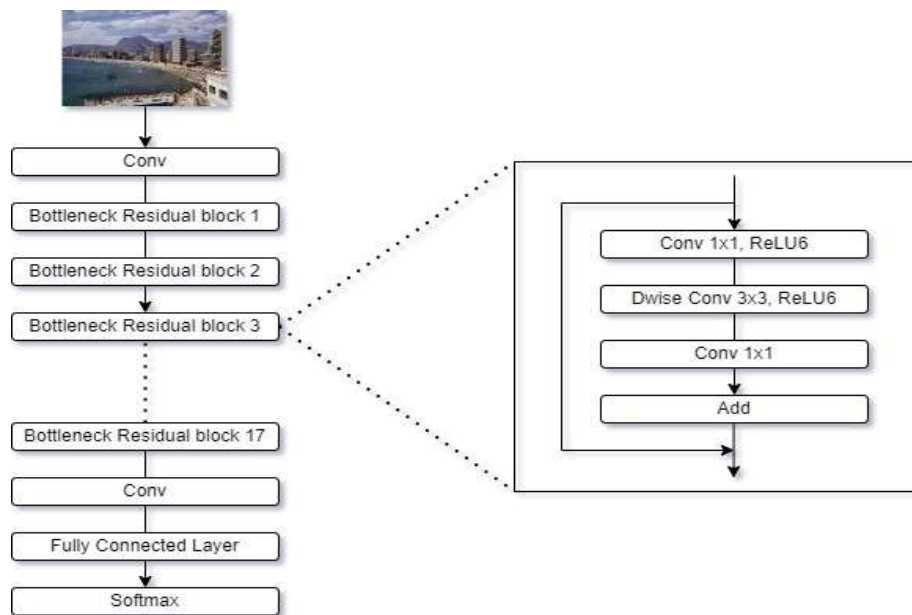


Fig. 7. Mobilenet architecture

IV. EXPERIMENTAL SETUP:

The models were evaluated using multiple measures. The training history was saved to keep track of the training accuracy, testing accuracy, and value of loss throughout the whole training. At the completion of the training, ten different batches of 200 images were passed to the models to evaluate them on the basis of their accuracy and confusion matrix. The formula for calculating the accuracy, given the confusion matrix for the model, is given in (1). Wang dataset is used for evaluation of models.

$$\text{Accuracy} = \frac{\text{TruePositives} + \text{TrueNegatives}}{\text{TruePositives} + \text{TrueNegatives} + \text{FalsePositives} + \text{FalseNegatives}} \quad (1)$$

The Wang dataset is a collection of 1000 images that are divided into ten different categories, with each category containing 100 images [20]. It is a well-annotated dataset that provides a comprehensive set of high-quality images for each category and has been widely used for classification and retrieval purposes.

The categories included in the dataset are tribe, beach, monuments, buses, dinosaurs, elephants, roses, horses, mountains, and food. It is a popular benchmark dataset for image classification tasks and has been widely used in academic research to evaluate and compare different image classification algorithms. One thousand images are a small dataset size for many high-volume models capable of storing high resolution and generalizing large feature sets from data. Therefore, image augmentation has been performed to increase the size of data and to reduce overfitting. Fig. 8 shows a sample image from each category of the Wang dataset.

V. EXPERIMENTAL RESULTS

Wang dataset has been used to evaluate six pre-trained deep CNN architectures. Google Colab is used for training models faster. Google Colab is a cloud-based platform that provides a free Jupyter notebook environment. Here, its cost-free GPUs are used for faster training. The performances of the models have been evaluated by observing their learning curves as well as training and testing accuracy to get a better idea of overfitting. Here, a smaller batch size, i.e.32, images per batch is used, for faster model training.

The accuracy of the models over ten different testing samples was calculated, as shown in Table 1. Training accuracy, loss, precision, and recall of all the models has been shown in Table 2. Each unique sample consisted of 20 percent of the dataset, i.e., 200 images. Finally, the accuracies are averaged out to infer that Densenet performs better with an average accuracy of 99.65%, followed closely by Xception and Inception. One hundred layers of the Densenet model are frozen for training, i.e., made untrainable. This was done because of the small dataset size and the high number of trainable parameters. The 100 layers of the models stayed trained on the imagenet dataset, and the rest of the layers were used for fine-tuning. The same pattern was followed while training the rest of the models by keeping approximately 80% of the total layers frozen and training the weights of the rest.



Fig. 8. Wang Dataset Sample Images [20]

TABLE I. ACCURACY OF PRE-TRAINED MODELS

	VGG16	Resnet50	Xception	Inception	Mobilenet	Densenet
Batch1	77.99%	90.50%	99.50%	99.50%	92.50%	100.00%
Batch 2	80.50%	88.50%	100.00%	100.00%	91.00%	99.50%
Batch 3	80.00%	91.00%	99.50%	100.00%	90.00%	99.00%
Batch 4	81.49%	93.00%	99.00%	99.50%	92.50%	100.00%
Batch 5	80.50%	86.00%	100.00%	99.50%	90.00%	99.00%
Batch 6	76.49%	88.00%	99.50%	100.00%	91.00%	100.00%
Batch 7	78.50%	90.00%	99.50%	99.50%	92.00%	100.00%
Batch 8	79.00%	89.50%	99.50%	99.50%	93.50%	99.50%
Batch 9	78.50%	88.00%	100.00%	98.50%	90.50%	100.00%
Batch 10	76.49%	86.00%	99.50%	99.50%	90.20%	99.50%
Average	78.94%	89.05%	99.60%	99.55%	91.32%	99.65%

TABLE II. TRAINING ACCURACY, LOSS, PRECISION, AND RECALL OF PRE-TRAINED MODELS

	VGG16	Resnet50	Xception	Inception	Mobilenet	Densenet
Training Accuracy	83.88%	91.50%	98.49%	99.87%	96.88%	99.67%
Training Loss	0.4668	0.2510	0.0877	0.0425	0.1906	0.0530
Precision	86.75%	91.70%	98.49%	98.87%	97.11%	99.00%
Recall	81.00%	91.12%	98.12%	98.87%	96.75%	99.00%

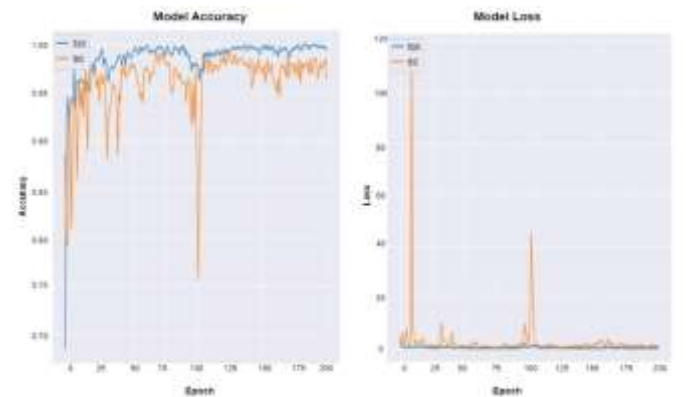


Fig. 9. Accuracy and loss curves for the Densenet model

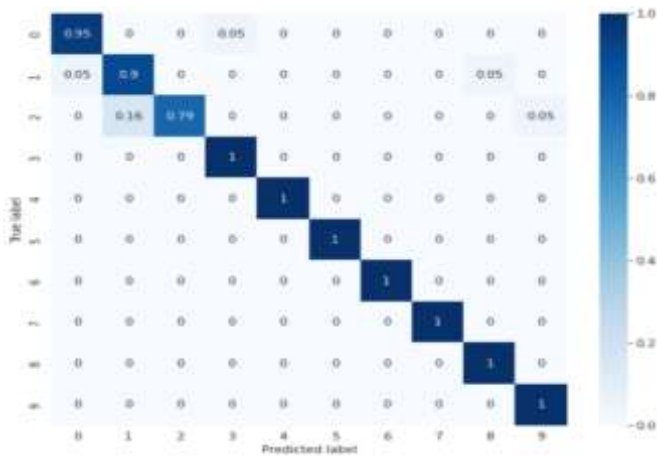


Fig. 10. Confusion matrix for Densenet

The learning curves for Densenet have been shown in Fig. 9. The curves show major fluctuations for about 100 epochs, but as the training progresses, they get smoothened out. The low difference in training and testing accuracies throughout the training shows that the model gives a good fit on the dataset. After experimenting with various numbers of epochs for training, it was observed that the performance of almost all models improved when trained for 200 epochs.

The confusion matrix is shown in Fig.10 provides the evaluation for all the classes. The model worked almost perfectly for all the classes but got confused between monuments and beaches. The reason for this could be similar visual features, such as color, texture, and shape. For example, a monument with a sandy-colored exterior may look identical to a beach with a matching color and texture of sand.

VI. CONCLUSION

This research has presented transfer learning on deep convolutional networks for six different architectures. Comparative analysis in terms of training and testing accuracies throughout the training as well as the final evaluation of the trained models, has been performed. Densenet-121 outperforms all the other models by giving an average testing accuracy of 99.65%. Xception and Inception-V3 have been known to outperform densenet in many scenarios, but for our particular use case where the dataset size is small, densenet gives the perfect fit. The number of non-trainable layers in densenet were set to 20 to leave more scope for training on the new data. Densenet due its number of trainable parameters, (7,628,484) turns out to have the perfect model volume for a dataset of our size, giving the better fit as compared to other models considered.

REFERENCES

- [1] R. Das, S. De, S. Bhattacharyya, J. Platos, V. Snasel, and A. E. Hassanien, "Data Augmentation and Feature Fusion for Melanoma Detection with Content Based Image Classification," 2020, pp. 712–721. doi: 10.1007/978-3-030-14118-9_70.
- [2] J. H. Dewan and S. D. Thepade, "How Scopus is Shaping the Research Publications of Feature Fusion-Based Image Retrieval," *Libr. Philos. Pract.*, vol. 2021, pp. 2–15, 2021.
- [3] R. Das, S. Thepade, and S. Ghosh, "Novel feature extraction technique for content-based image recognition with query classification," *Int. J. Comput. Vis. Robot.*, vol. 7, no. 1/2, p. 123, 2017, doi: 10.1504/IJCVR.2017.081240.
- [4] J. H. Dewan and S. D. Thepade, "Image Retrieval using Weighted Fusion of GLCM and TSBTC Features," in *2021 6th International*

Conference for Convergence in Technology (I2CT), Apr. 2021, pp. 1–7. doi: 10.1109/I2CT51068.2021.9418131.

- [5] A. KP and J. Anitha, "Plant disease classification using deep learning," in *2021 3rd International Conference on Signal Processing and Communication (ICSPC)*, May 2021, pp. 407–411. doi: 10.1109/ICSPC51351.2021.9451696.
- [6] R. M. Prakash and R. S. S. Kumari, "Classification of MR Brain Images for Detection of Tumor with Transfer Learning from Pre-trained CNN Models," in *2019 International Conference on Wireless Communications Signal Processing and Networking (WiSPNET)*, Mar. 2019, pp. 508–511. doi: 10.1109/WiSPNET45539.2019.9032811.
- [7] A. Kensert, P. J. Harrison, and O. Spjuth, "Transfer Learning with Deep Convolutional Neural Networks for Classifying Cellular Morphological Changes," *SLAS Discov.*, vol. 24, no. 4, pp. 466–475, Apr. 2019, doi: 10.1177/2472555218818756.
- [8] K. K. Anilkumar, V. J. Manoj, and T. M. Sagi, "Automated detection of leukemia by pretrained deep neural networks and transfer learning: A comparison," *Med. Eng. Phys.*, vol. 98, pp. 8–19, Dec. 2021, doi: 10.1016/j.medengphy.2021.10.006.
- [9] I. Oztel, G. Yolcu, and C. Oz, "Performance Comparison of Transfer Learning and Training from Scratch Approaches for Deep Facial Expression Recognition," in *2019 4th International Conference on Computer Science and Engineering (UBMK)*, Sep. 2019, pp. 1–6. doi: 10.1109/UBMK.2019.8907203.
- [10] M. F. Aydogdu, V. Celik, and M. F. Demirci, "Comparison of Three Different CNN Architectures for Age Classification," in *2017 IEEE 11th International Conference on Semantic Computing (ICSC)*, 2017, pp. 372–377. doi: 10.1109/ICSC.2017.61.
- [11] M. Jogin, Mohana, M. S. Madhulika, G. D. Divya, R. K. Meghana, and S. Apoorva, "Feature Extraction using Convolution Neural Networks (CNN) and Deep Learning," in *2018 3rd IEEE International Conference on Recent Trends in Electronics, Information & Communication Technology (RTEICT)*, May 2018, pp. 2319–2323. doi: 10.1109/RTEICT42901.2018.9012507.
- [12] C. Iorga and V.-E. Neagoe, "A Deep CNN Approach with Transfer Learning for Image Recognition," in *2019 11th International Conference on Electronics, Computers and Artificial Intelligence (ECAI)*, Jun. 2019, pp. 1–6. doi: 10.1109/ECAI46879.2019.9042173.
- [13] M. Shaha and M. Pawar, "Transfer Learning for Image Classification," in *2018 Second International Conference on Electronics, Communication and Aerospace Technology (ICECA)*, Mar. 2018, pp. 656–660. doi: 10.1109/ICECA.2018.8474802.
- [14] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," Sep. 2014, doi: 10.48550/arXiv.1409.1556.
- [15] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," Dec. 2015, [Online]. Available: <http://arxiv.org/abs/1512.03385>
- [16] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the Inception Architecture for Computer Vision," Dec. 2015, [Online]. Available: <http://arxiv.org/abs/1512.00567>
- [17] F. Chollet, "Xception: Deep Learning with Depthwise Separable Convolutions," Oct. 2016, [Online]. Available: <http://arxiv.org/abs/1610.02357>
- [18] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, "Densely Connected Convolutional Networks," Aug. 2016, [Online]. Available: <http://arxiv.org/abs/1608.06993>
- [19] A. G. Howard *et al.*, "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," Apr. 2017, [Online]. Available: <http://arxiv.org/abs/1704.04861>
- [20] Jia Li and J. Z. Wang, "Automatic linguistic indexing of pictures by a statistical modeling approach," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 25, no. 9, pp. 1075–1088, Sep. 2003, doi: 10.1109/TPAMI.2003.1227984.